

Bài 20: Annotation trong Java

Annotation là metadata dùng để cung cấp thông tin bổ sung cho class, method, field, parameter hoặc các thành phần khác trong chương trình Java.

Annotation không trực tiếp thay đổi dữ liệu của object, nhưng có thể được compiler, JVM hoặc framework đọc để kiểm tra và thực hiện hành vi tương ứng.

1. Annotation là gì?

Annotation bắt đầu bằng ký hiệu @.

Một số annotation quen thuộc:

```
@Override
@Deprecated
@SuppressWarnings("unchecked")
```

Ví dụ:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

@Override thông báo cho compiler rằng method sound đang ghi đè method của class cha.

2. Vai trò của Annotation

Annotation thường được dùng để:

- Cung cấp thông tin cho compiler.
- Kiểm tra code trong quá trình biên dịch.
- Cấu hình hành vi cho framework.
- Đánh dấu class hoặc method.
- Đọc metadata bằng Reflection.
- Sinh code tự động thông qua annotation processor.

3. Các annotation có sẵn

@Override

Kiểm tra method có thực sự override method của superclass hoặc interface hay không.

```
class Parent {
    void display() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child");
    }
}
```

Nếu viết sai tên method, compiler sẽ báo lỗi vì không có method phù hợp để override.

@Deprecated

Đánh dấu thành phần không nên tiếp tục sử dụng.

```
class OldApi {
    @Deprecated
    void oldMethod() {
        System.out.println("API cũ");
    }
}
```

Có thể cung cấp lý do và phiên bản thay thế:

```
/**
 * @deprecated Dùng newMethod() thay thế.
 */
@Deprecated(since = "2.0", forRemoval = true)
void oldMethod() {
}
```

@SuppressWarnings

Tắt một số cảnh báo của compiler.

```
@SuppressWarnings("unchecked")
void process() {
    // Code có cảnh báo unchecked
}
```

Không nên lạm dụng annotation này vì có thể che giấu lỗi thật.

@FunctionalInterface

Đảm bảo interface chỉ có một abstract method.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Nếu thêm abstract method thứ hai, compiler sẽ báo lỗi.

4. Tạo Custom Annotation

Dùng từ khóa @interface để tạo annotation riêng.

```
@interface Author {
    String name();
}
```

Sử dụng:

```
@Author(name = "Nguyen Van An")
class Student {
}
```

Một annotation có thể có nhiều thuộc tính:

```
@interface Info {
    String author();
    int version() default 1;
    String description() default "";
}
```

```
@Info(  
    author = "An",  
    version = 2,  
    description = "Class quản lý sinh viên"  
)  
class StudentService {  
}
```

5. Annotation Element

Các kiểu dữ liệu thường được phép dùng làm giá trị annotation:

- Kiểu nguyên thủy như int, boolean, double.
- String.
- Class.
- enum.
- Annotation khác.
- Mảng của các kiểu trên.

```
enum Role {  
    ADMIN,  
    USER  
}  
  
@interface UserConfig {  
    String username();  
    Role role();  
    boolean active() default true;  
    String[] permissions() default {};  
}
```

```
@UserConfig(  
    username = "an",  
    role = Role.ADMIN,  
    permissions = {"READ", "WRITE"}  
)  
class User {  
}
```

6. @Target

@Target quy định annotation được phép đặt ở đâu.

```
import java.lang.annotation.ElementType;  
import java.lang.annotation.Target;
```

```
@Target(ElementType.TYPE)
@interface Entity {
}
```

Annotation trên chỉ được dùng cho class, interface, enum hoặc record.

Các giá trị thường gặp:

ElementType	Vị trí sử dụng
TYPE	Class, interface, enum, record
METHOD	Method
FIELD	Field
PARAMETER	Parameter
CONSTRUCTOR	Constructor
LOCAL_VARIABLE	Biến local
PACKAGE	Package
ANNOTATION_TYPE	Một annotation khác
TYPE_USE	Vị trí sử dụng kiểu dữ liệu

Có thể cho phép nhiều vị trí:

```
@Target({ElementType.TYPE, ElementType.METHOD})
@interface Loggable {
}
```

7. @Retention

@Retention quy định annotation tồn tại đến thời điểm nào.

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)
@interface Description {
    String value();
}
```

Các mức retention:

Chính sách	Ý nghĩa
SOURCE	Chỉ tồn tại trong mã nguồn
CLASS	Có trong file .class nhưng thường không đọc được lúc runtime
RUNTIME	Có thể đọc bằng Reflection lúc chương trình chạy

Nếu muốn đọc annotation bằng Reflection, cần dùng `RetentionPolicy.RUNTIME`.

8. Đọc Annotation bằng Reflection

Ví dụ annotation:

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)
@interface Table {
    String name();
}
```

Gắn annotation vào class:

```
@Table(name = "students")
class Student {
}
```

Đọc annotation:

```
public class Main {
    public static void main(String[] args) {
        Class<Student> clazz = Student.class;

        if (clazz.isAnnotationPresent(Table.class)) {
            Table table = clazz.getAnnotation(Table.class);
            System.out.println(table.name()); // students
        }
    }
}
```

Reflection cho phép chương trình kiểm tra metadata của class trong lúc chạy.

9. Đọc Annotation của Method

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@interface Execute {
}
```

```
class Task {
    @Execute
    public void run() {
        System.out.println("Task đang chạy");
    }
}
```

```
for (Method method : Task.class.getDeclaredMethods()) {
    if (method.isAnnotationPresent(Execute.class)) {
        System.out.println("Tìm thấy method: " + method.getName());
    }
}
```

10. Annotation có thuộc tính value

Nếu annotation chỉ có một thuộc tính tên là value, có thể bỏ tên thuộc tính khi sử dụng.

```
@interface Column {
    String value();
}
```

Hai cách viết sau tương đương:

```
@Column(value = "student_name")
class Student {
}
```

```
@Column("student_name")
class Student {
}
```

11. Repeatable Annotation

Một annotation có thể được sử dụng nhiều lần trên cùng một thành phần nếu được đánh dấu bằng `@Repeatable`.

```
import java.lang.annotation.Repeatable;

@Repeatable(Roles.class)
@interface Role {
    String value();
}

@interface Roles {
    Role[] value();
}
```

Sử dụng:

```
@Role("ADMIN")
@Role("EDITOR")
class User {
}
```

12. Annotation trong Framework

Annotation được sử dụng rất nhiều trong các framework Java.

Ví dụ với Spring:

```
@Component
public class UserService {
}
```

Ví dụ với JPA:

```
@Entity
@Table(name = "students")
public class Student {
}
```

Ví dụ với JUnit:

```
@Test
void shouldAddTwoNumbers() {
}
```

Framework đọc annotation và thực hiện hành vi tương ứng.

13. Annotation không tự thực hiện hành vi

Annotation chỉ là metadata. Nó không tự động chạy logic nếu không có compiler, JVM, framework hoặc code Reflection xử lý nó.

```
@Author(name = "An")
class Student {
}
```

Muốn in tên tác giả, cần đọc annotation:

```
Author author = Student.class.getAnnotation(Author.class);
System.out.println(author.name());
```

14. Điểm chính

- Annotation bắt đầu bằng ký hiệu @.
- Annotation cung cấp metadata cho compiler, JVM hoặc framework.
- Dùng @interface để tạo custom annotation.
- @Target xác định vị trí annotation được sử dụng.
- @Retention xác định thời gian tồn tại của annotation.
- Dùng RetentionPolicy.RUNTIME nếu cần đọc bằng Reflection.
- Annotation không tự thực hiện hành vi nếu không có code xử lý.
- @Override, @Deprecated, @SuppressWarnings và @FunctionalInterface là các annotation phổ biến.

15. Thực hành tốt

- Đặt tên annotation rõ nghĩa.
- Chỉ cho phép vị trí cần thiết bằng @Target.
- Dùng giá trị mặc định hợp lý cho thuộc tính.
- Dùng RUNTIME khi annotation cần được đọc lúc chạy.
- Không lạm dụng Reflection vì có thể làm code khó theo dõi.
- Không dùng @SuppressWarnings để che giấu lỗi chưa hiểu rõ.
- Viết tài liệu cho custom annotation có ảnh hưởng đến framework.

16. Các lỗi thường gặp

Lỗi	Nguyên nhân
Annotation không đọc được bằng Reflection	Thiếu @Retention(RetentionPolicy.RUNTIME)
Annotation đặt sai vị trí	@Target không cho phép vị trí đó
Thiếu thuộc tính bắt buộc	Annotation element không có default
Dùng nhiều lần nhưng không có @Repeatable	Annotation không hỗ trợ lặp
Nghĩ annotation tự chạy logic	Chưa có code hoặc framework xử lý
@Override báo lỗi	Method không thực sự override method của class cha

17. Ghi nhớ

Annotation = metadata gắn vào code

@Target → Annotation được đặt ở đâu?
@Retention → Annotation tồn tại đến khi nào?
Reflection → Đọc annotation lúc runtime

Ví dụ tổng quát:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface EntityInfo {
    String name();
}

@EntityInfo(name = "Student")
class Student {
}
```